# Stack

---

## 🧠 1. Stack Kya Hota Hai?

**Stack = Linear Data Structure**

Jo follow karta hai: **LIFO (Last In First Out)**

Matlab jo last me dala, wahi sabse pehle niklega.

Real Life Example:

- Plates ka stack
- Browser back button
- Function call stack

---

## 🧩 2. LIFO Principle

```
Push 10
Push 20
Push 30

Stack top -> 30

Pop -> 30 remove
Pop -> 20 remove
Pop -> 10 remove
```

Order of removal = reverse of insertion.

---

## ⚙️ 3. Basic Operations

| Operation | Meaning | Time Complexity |
|---|---|---|
| push(x) | element add karo top pe | O(1) |
| pop() | top element remove karo | O(1) |

| Operation | Meaning | Time Complexity |
|---|---|---|
| top()/peek() | top element dekho | O(1) |
| isEmpty() | stack empty hai kya | O(1) |
| isFull() | stack full hai kya (array case) | O(1) |

👉 Sab operations constant time me hote hain.

---

# 🔥 4. Stack Implementation Using Array (From Scratch)

FAANG interview me agar bole: “Implement your own stack” — ye karna hai.

---

## 🟢 Java Implementation (Array Based Stack)

```
class StackArray {

    private int[] arr;   // Array to store stack elements
    private int top;     // Index of top element
    private int capacity; // Maximum size of stack

    // Constructor
    public StackArray(int size) {
        capacity = size;          // Initialize capacity
        arr = new int[capacity];  // Create array
        top = -1;                 // Initially stack is emp
ty
    }

    // Check if stack is empty
    public boolean isEmpty() {
        return top == -1;   // If top is -1, no elements
    }

    // Check if stack is full
    public boolean isFull() {
        return top == capacity - 1;  // If top reaches last
index
```

```
    }

    // Push operation
    public void push(int value) {
        if (isFull()) {
            System.out.println("Stack Overflow! Cannot push
" + value);
            return;
        }

        top++;              // Move top pointer up
        arr[top] = value;   // Insert value at new top
    }

    // Pop operation
    public int pop() {
        if (isEmpty()) {
            System.out.println("Stack Underflow! Stack is e
mpty");
            return -1;
        }

        int popped = arr[top]; // Store top value
        top--;                 // Move top pointer down
        return popped;         // Return popped value
    }

    // Peek operation
    public int peek() {
        if (isEmpty()) {
            System.out.println("Stack is empty");
            return -1;
        }

        return arr[top];  // Return top element
    }

    // Print stack
```

```
    public void printStack() {
        for (int i = top; i >= 0; i--) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
    }
}
```

---

### 🧠 Important Interview Points

- `top = -1` means empty stack
- Array stack has fixed size → Overflow possible
- All operations O(1)

---

# 🔵 5. Stack Using Linked List

Advantage:

- No overflow until memory full
- Dynamic size

---

## Java Implementation (Linked List Stack)

```
class StackLinkedList {

    // Node class
    class Node {
        int data;
        Node next;

        Node(int data) {
            this.data = data;
            this.next = null;
        }
    }
```

```
    private Node top; // Top pointer

    public StackLinkedList() {
        top = null;  // Initially empty
    }

    // Check empty
    public boolean isEmpty() {
        return top == null;
    }

    // Push
    public void push(int value) {
        Node newNode = new Node(value);

        newNode.next = top;  // New node points to old top
        top = newNode;       // Update top
    }

    // Pop
    public int pop() {
        if (isEmpty()) {
            System.out.println("Stack Underflow");
            return -1;
        }

        int popped = top.data;  // Store top data
        top = top.next;         // Move top
        return popped;
    }

    // Peek
    public int peek() {
        if (isEmpty()) {
            System.out.println("Stack Empty");
            return -1;
        }
```

```
        return top.data;
    }
}
```

---

# 🟣 6. Inbuilt Stack

### Java:

```
import java.util.Stack;

Stack<Integer> stack = new Stack<>();
```

### C++:

```
#include <stack>
stack<int> st;
```

⚠️ Interview me inbuilt allowed hota hai — but concept clear hona zaruri hai.

---

# 🧪 7. Practice Problem 1: Reverse a String Using Stack

## Idea:

- String ke characters stack me push karo
- Fir pop karke new string banao
- LIFO → string reverse ho jayegi

---

```
import java.util.Stack;

public class ReverseString {

    public static String reverse(String str) {
```

```
        Stack<Character> stack = new Stack<>();

        // Step 1: Push all characters
        for (int i = 0; i < str.length(); i++) {
            stack.push(str.charAt(i));
        }

        // Step 2: Pop and build reversed string
        String reversed = "";

        while (!stack.isEmpty()) {
            reversed += stack.pop();
        }

        return reversed;
    }

    public static void main(String[] args) {
        String input = "FAANG";
        System.out.println(reverse(input));  // GNAAF
    }
}
```

Time Complexity: O(n)

Space Complexity: O(n)

---

# 🧪 8. Practice Problem 2: Balanced Parentheses

Very Important FAANG Question.

---

## Problem:

Check if string like:

```
"()[]{}"  -> valid
```

```
"(]"      -> invalid
```

---

## Logic:

1. Opening bracket → push
2. Closing bracket:
   - If stack empty → invalid
   - If top matches → pop
   - Else → invalid
3. End me stack empty hona chahiye

---

## Java Code (FAANG Level Clean Code)

```
import java.util.Stack;

public class BalancedParentheses {

    public static boolean isValid(String s) {

        Stack<Character> stack = new Stack<>();

        for (int i = 0; i < s.length(); i++) {

            char ch = s.charAt(i);

            // If opening bracket, push into stack
            if (ch == '(' || ch == '{' || ch == '[') {
                stack.push(ch);
            }

            // If closing bracket
            else {

                // If stack empty -> no matching opening
                if (stack.isEmpty()) {
```

```
                    return false;
                }

                char top = stack.pop();

                // Check matching
                if (ch == ')' && top != '(') return false;
                if (ch == '}' && top != '{') return false;
                if (ch == ']' && top != '[') return false;
            }
        }

        // If stack empty -> valid
        return stack.isEmpty();
    }

    public static void main(String[] args) {
        System.out.println(isValid("()[]{}"));  // true
        System.out.println(isValid("("));      // false
    }
}
```

Time Complexity: O(n)

Space Complexity: O(n)

---

# 🎯 FAANG Level Thinking

Stack Use Cases:

- Balanced parentheses
- Function call stack
- Undo/Redo
- Next Greater Element
- Monotonic Stack
- Expression Evaluation
- Largest Rectangle in Histogram

# 🏆 Interview Me Ye Bolna

> Stack is a linear data structure that follows LIFO principle.
>
> All operations (push, pop, peek) work in O(1).
>
> It can be implemented using array or linked list.
>
> It is heavily used in recursion, expression evaluation, monotonic problems, and parsing.

---

# 🧠 CORE PATTERN UNDERSTANDING

Parenthesis problems 3 categories me aati hain:

---

## ✅ 1️⃣ Bracket Matching (Classic Stack)

👉 Jab **exact matching** karna ho

Example:

- Valid Parentheses
- Remove Invalid Parentheses

Signal:

> "Check if valid"
>
> "Matching brackets"
>
> Multiple bracket types

Tool:

👉 Stack

---

## ✅ 2️⃣ Open-Close Counting (Balance Tracking)

👉 Jab sirf `(` aur `)` ho

Example:

- Minimum Add to Make Valid

- Minimum Remove to Make Valid

Signal:

> Count needed
>
> Balance maintain karna
>
> Stack optional

Tool:

👉 Counter or Stack

---

## ✅ 3️⃣ Longest Valid / Depth / Continuous Length

👉 Jab longest length ya nesting depth poochi jaaye

Example:

- Longest Valid Parentheses
- Maximum Nesting Depth

Tool:

👉 Stack with indices OR DP

---

# 🔥 1️⃣ Valid Parentheses (LeetCode 20)

## 🧠 Pattern

- Opening → push
- Closing → match & pop
- End me stack empty hona chahiye

---

## ⭐ Optimized Java Code

```
import java.util.Stack;

class Solution {
    public boolean isValid(String s) {
```

```
        Stack<Character> stack = new Stack<>();

        for (int i = 0; i < s.length(); i++) {

            char ch = s.charAt(i);

            // Opening brackets -> push
            if (ch == '(' || ch == '{' || ch == '[') {
                stack.push(ch);
            }

            else {

                // No opening bracket available
                if (stack.isEmpty()) return false;

                char top = stack.pop();

                // Check matching
                if (ch == ')' && top != '(') return false;
                if (ch == '}' && top != '{') return false;
                if (ch == ']' && top != '[') return false;
            }
        }

        return stack.isEmpty();
    }
}
```

Time: O(n)

Space: O(n)

---

# 🔥 2️⃣ Minimum Add to Make Parentheses Valid (LC 921)

Example:

```
"()))((
Answer = 4
```

---

## 🧠 Pattern Thinking

Maintain:

- `balance` → open brackets count
- `additions` → kitne add karne padenge

Rules:

- `(` → balance++
- `)`:
  - If balance > 0 → balance--
  - Else → additions++

End me:

Total = additions + balance

---

## ⭐ Optimal Java Code (NO STACK Needed)

```
class Solution {
    public int minAddToMakeValid(String s) {

        int balance = 0;   // Tracks unmatched '('
        int additions = 0; // Tracks needed '('

        for (char ch : s.toCharArray()) {

            if (ch == '(') {
                balance++;  // Opening bracket
            }
            else {

                if (balance > 0) {
                    balance--; // Match found
```

```
                } else {
                    additions++; // Need an extra '('
                }
            }
        }

        // Remaining unmatched '(' also need ')'
        return additions + balance;
    }
}
```

Time: O(n)

Space: O(1)

🔥 Interview me bolna:

> We don't need stack because we only track count.

---

# 🔥 3️⃣ Remove Invalid Parentheses (LC 1249 Basic Version)

👉 Remove minimum invalid parentheses to make valid.

---

## 🧠 Pattern

Use stack to track invalid indices.

Steps:

1. Traverse
2. Store unmatched '(' indices
3. Mark invalid ')'
4. Remove all invalid positions

---

## ⭐ Clean Java Code

```
import java.util.*;

class Solution {
    public String minRemoveToMakeValid(String s) {

        Stack<Integer> stack = new Stack<>();
        Set<Integer> remove = new HashSet<>();

        for (int i = 0; i < s.length(); i++) {

            char ch = s.charAt(i);

            if (ch == '(') {
                stack.push(i);
            }

            else if (ch == ')') {

                if (!stack.isEmpty()) {
                    stack.pop();
                } else {
                    remove.add(i); // Invalid ')'
                }
            }
        }

        // Remaining unmatched '('
        while (!stack.isEmpty()) {
            remove.add(stack.pop());
        }

        // Build result
        StringBuilder result = new StringBuilder();

        for (int i = 0; i < s.length(); i++) {
            if (!remove.contains(i)) {
                result.append(s.charAt(i));
            }
```

```
        }

        return result.toString();
    }
}
```

Time: O(n)

Space: O(n)

---

# 🔥 4️⃣ Longest Valid Parentheses (LC 32) ⭐⭐ VERY IMPORTANT

Most students fail here.

---

## 🧠 Core Idea

Stack me indices store karo, characters nahi.

WHY?

Because length calculate karna hai.

---

## Trick:

1. Stack me initially push `1`
2. Jab '(' → push index
3. Jab ')':
    - Pop
    - If empty → push current index
    - Else → length = i - stack.peek()

---

## ⭐ Optimized Java Code

```
import java.util.Stack;

class Solution {
```

```
    public int longestValidParentheses(String s) {

        Stack<Integer> stack = new Stack<>();

        stack.push(-1); // Base for length calculation

        int maxLength = 0;

        for (int i = 0; i < s.length(); i++) {

            if (s.charAt(i) == '(') {
                stack.push(i);
            }
            else {

                stack.pop();

                if (stack.isEmpty()) {
                    stack.push(i); // Reset base
                }
                else {
                    maxLength = Math.max(maxLength, i - sta
ck.peek());
                }
            }
        }

        return maxLength;
    }
}
```

Time: O(n)

Space: O(n)

---

# 🎯 Pattern Recognition Table

| Problem | Tool | Why |
|---|---|---|
| Valid Parentheses | Stack | Matching |
| Minimum Add | Counter | Just balance tracking |
| Remove Invalid | Stack + Set | Index removal |
| Longest Valid | Stack (index) | Length calculation |

---

# 🧠 Interview Master Insight

Parenthesis problems solve by asking:

1. Matching chahiye?

   → Stack

2. Count hi chahiye?

   → Counter

3. Longest length?

   → Stack with indices

4. Minimum remove?

   → Track invalid positions

---

# 🏆 FAANG Ready Thinking Line

> Parenthesis problems can be categorized into matching, counting, and longest-valid patterns.
>
> Based on requirement, we either use stack for structure tracking or counters for balance tracking.

---

---

# 🧠 1️⃣ Monotonic Stack Kya Hota Hai?

👉 Aisa stack jo **hamesha sorted order maintain karta hai**

Do types:

## 🔵 Monotonic Increasing Stack

Stack elements bottom → top increasing order me hote hain.

Example:

```
[1, 3, 5, 8]
```

---

### 🔴 Monotonic Decreasing Stack

Stack elements bottom → top decreasing order me hote hain.

Example:

```
[9, 6, 4, 2]
```

---

## 🔥 WHY IMPORTANT?

Ye pattern use hota hai jab problem bole:

- Next Greater
- Next Smaller
- Previous Greater
- Previous Smaller
- First bigger element
- Nearest smaller element

Signal word:

👉 "Next"

👉 "Previous"

👉 "Nearest"

👉 "First"

---

## 🎯 CORE INTUITION

For each element:

- Hum stack maintain karte hain

- Jab current element rule violate kare

  → pop until valid

---

# ⚡ Golden Rule

## If finding Next Greater Element (NGE)

👉 Use **Monotonic Decreasing Stack**

Why?

Kyuki jab bada element aaye

To chhote sabko pop karega.

---

# 📌 1️⃣ Next Greater Element (NGE)

Problem:

For each element, find next element greater than it.

Example:

```
[2,1,2,4,3]

Answer:
[4,2,4,-1,-1]
```

---

## 🧠 Pattern

Traverse from RIGHT to LEFT.

Maintain:

Monotonic Decreasing Stack

Steps:

1. While stack not empty AND stack.peek() <= arr[i]

   → pop

2. If stack empty → answer = -1

3. Else → answer = stack.peek()
4. Push current element

---

## ⭐ Java Code (FAANG Clean Version)

```
import java.util.Stack;

class Solution {
    public int[] nextGreaterElement(int[] nums) {

        int n = nums.length;
        int[] result = new int[n];
        Stack<Integer> stack = new Stack<>();

        // Traverse from right to left
        for (int i = n - 1; i >= 0; i--) {

            // Remove all smaller or equal elements
            while (!stack.isEmpty() && stack.peek() <= nums
[i]) {
                stack.pop();
            }

            // If stack empty, no greater element
            if (stack.isEmpty()) {
                result[i] = -1;
            } else {
                result[i] = stack.peek();
            }

            // Push current element
            stack.push(nums[i]);
        }

        return result;
    }
}
```

Time: O(n)

Space: O(n)

🔥 Important:

Each element pushed once, popped once → O(n)

---

# 📌 2️⃣ Next Smaller Element (NSE)

Same logic, just reverse condition.

---

## 🧠 Pattern

Use **Monotonic Increasing Stack**

While:

```
stack.peek() >= current
```

→ pop

---

## ⭐ Code

```
public int[] nextSmallerElement(int[] nums) {

    int n = nums.length;
    int[] result = new int[n];
    Stack<Integer> stack = new Stack<>();

    for (int i = n - 1; i >= 0; i--) {

        while (!stack.isEmpty() && stack.peek() >= nums[i])
{
            stack.pop();
        }

        if (stack.isEmpty()) {
            result[i] = -1;
        } else {
```

```
            result[i] = stack.peek();
        }

        stack.push(nums[i]);
    }

    return result;
}
```

---

# 📌 3️⃣ Previous Greater Element (PGE)

Ab LEFT side dekhna hai.

Traverse:

LEFT → RIGHT

Use:

Monotonic Decreasing Stack

---

## ⭐ Code

```
public int[] previousGreater(int[] nums) {

    int n = nums.length;
    int[] result = new int[n];
    Stack<Integer> stack = new Stack<>();

    for (int i = 0; i < n; i++) {

        while (!stack.isEmpty() && stack.peek() <= nums[i])
{
            stack.pop();
        }

        if (stack.isEmpty()) {
            result[i] = -1;
        } else {
```

```
            result[i] = stack.peek();
        }

        stack.push(nums[i]);
    }

    return result;
}
```

---

# 📌 4️⃣ Previous Smaller Element (PSE)

Traverse:

LEFT → RIGHT

Use:

Monotonic Increasing Stack

Condition:

```
while peek >= current
```

---

# 🧠 MEMORY TRICK (IMPORTANT)

| Problem | Stack Type | Traverse |
|---|---|---|
| NGE | Decreasing | Right → Left |
| NSE | Increasing | Right → Left |
| PGE | Decreasing | Left → Right |
| PSE | Increasing | Left → Right |

---

# 🔥 Why It's O(n) Not O($n^2$)?

Because:

Each element:

- pushed once

- popped once

Total operations ≤ 2n

---

# 🚀 Advanced Use Cases

Monotonic Stack used in:

- Largest Rectangle in Histogram
- Trapping Rain Water
- Daily Temperatures
- Stock Span Problem
- Sum of Subarray Minimums
- Remove K Digits
- Maximum Area Rectangle

---

# 🧠 Deep Insight (FAANG Level)

Monotonic stack works because:

👉 It maintains candidate answers

👉 Removes useless elements

👉 Guarantees nearest greater/smaller

---

# 🏆 Interview Ready Line

> Monotonic stack is used to efficiently solve nearest greater or smaller element problems in O(n) time by maintaining a sorted stack and removing irrelevant candidates dynamically.

---

---

# 1️⃣ Next Greater Element I (LC 496)

## 🧠 Problem

nums1 ke har element ka NGE find karo nums2 me.

Example:

```
nums1 = [4,1,2]
nums2 = [1,3,4,2]

Answer = [-1,3,-1]
```

---

## 🔥 Pattern

Step 1:

nums2 ka NGE precompute karo using Monotonic Decreasing Stack.

Step 2:

Map me store karo.

---

## ⭐ Code

```
import java.util.*;

class Solution {
    public int[] nextGreaterElement(int[] nums1, int[] nums
2) {

        Map<Integer, Integer> map = new HashMap<>();
        Stack<Integer> stack = new Stack<>();

        // Step 1: Compute NGE for nums2
        for (int i = nums2.length - 1; i >= 0; i--) {

            while (!stack.isEmpty() && stack.peek() <= nums
2[i]) {
                stack.pop();
            }

            int nge = stack.isEmpty() ? -1 : stack.peek();
            map.put(nums2[i], nge);
```

```
            stack.push(nums2[i]);
        }

        // Step 2: Build answer for nums1
        int[] result = new int[nums1.length];

        for (int i = 0; i < nums1.length; i++) {
            result[i] = map.get(nums1[i]);
        }

        return result;
    }
}
```

Time: O(n + m)

---

# 2️⃣ Next Greater Element II (Circular) (LC 503)

## 🧠 Trick

Circular array → array ko 2 baar traverse karo.

Use:

i % n

---

## ⭐ Code

```
class Solution {
    public int[] nextGreaterElements(int[] nums) {

        int n = nums.length;
        int[] result = new int[n];
        Arrays.fill(result, -1);

        Stack<Integer> stack = new Stack<>();
```

```
        // Traverse twice
        for (int i = 2 * n - 1; i >= 0; i--) {

            int num = nums[i % n];

            while (!stack.isEmpty() && stack.peek() <= num)
{
                stack.pop();
            }

            if (i < n) {
                result[i] = stack.isEmpty() ? -1 : stack.pe
ek();
            }

            stack.push(num);
        }

        return result;
    }
}
```

Time: O(n)

---

# 3️⃣ Daily Temperatures (LC 739)

## 🧠 Problem

Har din ke liye batana hai kitne din baad warmer temperature milega.

Example:

```
[73,74,75,71,69,72,76,73]
```

---

## 🔥 Pattern

Need:

Next Greater Element but return distance.

Store:

Indices in stack (not values)

---

## ⭐ Code

```
class Solution {
    public int[] dailyTemperatures(int[] temps) {

        int n = temps.length;
        int[] result = new int[n];

        Stack<Integer> stack = new Stack<>();

        for (int i = n - 1; i >= 0; i--) {

            while (!stack.isEmpty() && temps[stack.peek()]
<= temps[i]) {
                stack.pop();
            }

            if (stack.isEmpty()) {
                result[i] = 0;
            } else {
                result[i] = stack.peek() - i;
            }

            stack.push(i);
        }

        return result;
    }
}
```

---

# 4️⃣ Stock Span Problem

## 🧠 Problem

Kitne consecutive days tak stock price <= current price.

---

## 🔥 Pattern

Previous Greater Element ka variation.

Traverse:

Left → Right

---

## ⭐ Code

```
class StockSpanner {

    Stack<int[]> stack;
    // pair: [price, span]

    public StockSpanner() {
        stack = new Stack<>();
    }

    public int next(int price) {

        int span = 1;

        while (!stack.isEmpty() && stack.peek()[0] <= pric
e) {
            span += stack.pop()[1];
        }

        stack.push(new int[]{price, span});

        return span;
    }
}
```

Time per operation: Amortized O(1)

---

# 5️⃣ Largest Rectangle in Histogram (LC 84) ⭐⭐⭐ VERY IMPORTANT

## 🧠 Core Idea

For each bar:

Find:

- Previous Smaller
- Next Smaller

Area = height * width

Width = (rightSmall - leftSmall - 1)

---

## 🔥 Optimized One-Pass Stack Solution

```
class Solution {
    public int largestRectangleArea(int[] heights) {

        Stack<Integer> stack = new Stack<>();
        int maxArea = 0;
        int n = heights.length;

        for (int i = 0; i <= n; i++) {

            int height = (i == n) ? 0 : heights[i];

            while (!stack.isEmpty() && height < heights[sta
ck.peek()]) {

                int h = heights[stack.pop()];

                int rightBoundary = i;
                int leftBoundary = stack.isEmpty() ? -1 : s
tack.peek();

                int width = rightBoundary - leftBoundary -
1;
```

```
                maxArea = Math.max(maxArea, h * width);
            }

            stack.push(i);
        }

        return maxArea;
    }
}
```

Time: O(n)

🔥 This is top FAANG question.

---

# 6️⃣ Trapping Rain Water (Stack Version) (LC 42)

## 🧠 Idea

When current height > stack top height

→ we found a bounded container.

---

## ⭐ Code

```
class Solution {
    public int trap(int[] height) {

        Stack<Integer> stack = new Stack<>();
        int water = 0;

        for (int i = 0; i < height.length; i++) {

            while (!stack.isEmpty() && height[i] > height[s
tack.peek()]) {

                int bottom = stack.pop();
```

```
                if (stack.isEmpty()) break;

                int left = stack.peek();
                int width = i - left - 1;

                int boundedHeight = Math.min(height[i], hei
ght[left]) - height[bottom];

                water += width * boundedHeight;
            }

            stack.push(i);
        }

        return water;
    }
}
```

Time: O(n)

---

# 🏆 MASTER SUMMARY TABLE

| Problem | Stack Type | Store |
|---|---|---|
| NGE I | Decreasing | Value |
| NGE II | Decreasing | Value |
| Daily Temp | Decreasing | Index |
| Stock Span | Decreasing | Price + Span |
| Largest Rectangle | Increasing | Index |
| Rain Water | Increasing | Index |

---

# 🎯 FINAL FAANG INSIGHT

Monotonic Stack is used when:

- Nearest greater/smaller

- Span problems
- Range width calculation
- Area problems
- Water trapping problems

Core philosophy:

👉 Remove useless elements

👉 Maintain candidate answers

👉 Process each element once

---

# 🧠 1️⃣ Expression Types Samjho Pehle

## 1. Infix

Operator beech me

```
A + B
(3 + 4) * 5
```

Human readable

Computer ko difficult

---

## 2. Postfix (Reverse Polish Notation - RPN)

Operator end me

```
A B +
3 4 + 5 *
```

Computer friendly

No parentheses needed

---

## 3. Prefix (Polish Notation)

Operator start me

```
+ A B
* + 3 4 5
```

---

# 🧠 WHY Stack Required?

Because:

- Operator precedence handle karna
- Parentheses handle karna
- Order preserve karna

---

# 🎯 Operator Precedence

| Operator | Precedence |
|---|---|
| + - | 1 |
| * / | 2 |
| ^ | 3 |

---

# 🔥 1️⃣ Infix → Postfix

Example:

```
A + B
Output:
A B +
```

Example:

```
(A + B) * C
Output:
A B + C *
```

---

# 🧠 Algorithm

1. Operand → directly output
2. '(' → push
3. ')' → pop until '('
4. Operator:
   - While stack not empty AND
   - precedence(top) >= precedence(current)

     → pop to output
   - Then push current operator

---

## ⭐ Java Code (Fully Explained)

```
import java.util.Stack;

class Solution {

    // Function to return precedence
    static int precedence(char op) {
        if (op == '+' || op == '-') return 1;
        if (op == '*' || op == '/') return 2;
        if (op == '^') return 3;
        return -1;
    }

    public static String infixToPostfix(String exp) {

        StringBuilder result = new StringBuilder();
        Stack<Character> stack = new Stack<>();

        for (int i = 0; i < exp.length(); i++) {

            char ch = exp.charAt(i);

            // If operand, add to result
            if (Character.isLetterOrDigit(ch)) {
                result.append(ch);
```

```
            }

            // If '(' push to stack
            else if (ch == '(') {
                stack.push(ch);
            }

            // If ')', pop until '('
            else if (ch == ')') {

                while (!stack.isEmpty() && stack.peek() !=
'(') {
                    result.append(stack.pop());
                }

                stack.pop(); // Remove '('
            }

            // If operator
            else {

                while (!stack.isEmpty() &&
                        precedence(stack.peek()) >= precede
nce(ch)) {

                    result.append(stack.pop());
                }

                stack.push(ch);
            }
        }

        // Pop remaining operators
        while (!stack.isEmpty()) {
            result.append(stack.pop());
        }

        return result.toString();
```

```
    }
}
```

Time: O(n)

---

# 🔥 2️⃣ Infix → Prefix

Trick:

1. Reverse string
2. Swap '(' and ')'
3. Convert to postfix
4. Reverse result

---

## ⭐ Code

```
public static String infixToPrefix(String exp) {

    // Step 1: Reverse expression
    StringBuilder input = new StringBuilder(exp);
    input.reverse();

    // Step 2: Swap brackets
    for (int i = 0; i < input.length(); i++) {

        if (input.charAt(i) == '(')
            input.setCharAt(i, ')');
        else if (input.charAt(i) == ')')
            input.setCharAt(i, '(');
    }

    // Step 3: Convert to postfix
    String postfix = infixToPostfix(input.toString());

    // Step 4: Reverse postfix
```

```
    return new StringBuilder(postfix).reverse().toString();
}
```

---

# 🔥 3️⃣ Postfix Evaluation

Example:

```
2 3 +
```

Steps:

1. Operand → push
2. Operator → pop two operands
3. Apply
4. Push result

---

## ⭐ Java Code

```
import java.util.Stack;

public class PostfixEvaluation {

    public static int evaluatePostfix(String exp) {

        Stack<Integer> stack = new Stack<>();

        for (int i = 0; i < exp.length(); i++) {

            char ch = exp.charAt(i);

            // If digit, push to stack
            if (Character.isDigit(ch)) {
                stack.push(ch - '0');
            }

            else {
```

```
                int b = stack.pop();
                int a = stack.pop();

                switch (ch) {
                    case '+': stack.push(a + b); break;
                    case '-': stack.push(a - b); break;
                    case '*': stack.push(a * b); break;
                    case '/': stack.push(a / b); break;
                }
            }
        }

        return stack.pop();
    }
}
```

---

# 🔥 4️⃣ Prefix Evaluation

Trick:

Traverse from RIGHT → LEFT

---

## ⭐ Code

```
public static int evaluatePrefix(String exp) {

    Stack<Integer> stack = new Stack<>();

    for (int i = exp.length() - 1; i >= 0; i--) {

        char ch = exp.charAt(i);

        if (Character.isDigit(ch)) {
            stack.push(ch - '0');
        }
```

```
        else {

            int a = stack.pop();
            int b = stack.pop();

            switch (ch) {
                case '+': stack.push(a + b); break;
                case '-': stack.push(a - b); break;
                case '*': stack.push(a * b); break;
                case '/': stack.push(a / b); break;
            }
        }
    }

    return stack.pop();
}
```

---

# 🧠 FAANG Insight

Expression conversion uses:

👉 Stack for operators

👉 Output for operands

👉 Precedence handling

Evaluation uses:

👉 Stack for operands

👉 Operator applies on top two elements

---

# 🏗️ Applications

## 1️⃣ Expression Parsing

- Compiler syntax tree
- AST construction

## 2️⃣ Calculators

- Shunting Yard Algorithm
- Reverse Polish Notation

### 3️⃣ Compiler Design

- Intermediate Code Generation
- Parsing phase
- Operator precedence parsing

---

# 🏆 Interview Ready Summary

> Infix expressions require precedence handling and parentheses tracking, so we use a stack to temporarily store operators.
>
> Postfix and Prefix expressions eliminate parentheses and make evaluation easier using stack-based operand processing.

---

---

# 🧠 1️⃣ Stack Memory in Recursion

Sabse pehle ek myth clear karte hain:

👉 **Recursion internally stack hi use karta hai**

Isliye kehte hain:

> Recursion = implicit stack
>
> Stack DS = explicit stack

---

## 🔥 Example

```
void fun(int n) {
    if (n == 0) return;
    System.out.println(n);
    fun(n - 1);
}
```

Call:

```
fun(3)
```

---

## 📦 Call Stack Visualization

Step by step:

```
fun(3)
  |
  → fun(2)
        |
        → fun(1)
              |
              → fun(0)
```

Memory me stack frames:

```
| fun(3) |
| fun(2) |
| fun(1) |
| fun(0) |
```

Jab base case hit hota hai, unwind hota hai:

```
fun(0) returns
fun(1) returns
fun(2) returns
fun(3) returns
```

---

# 🧠 2️⃣ Function Call Stack Kya Store Karta Hai?

Har stack frame store karta hai:

- Local variables
- Parameters

- Return address
- Function state

Isliye recursion me space complexity hoti hai:

```
O(recursion depth)
```

---

# 🧠 IMPORTANT

Recursion me do phases hote hain:

1️⃣ Going Down (Call Phase)

2️⃣ Coming Back (Return Phase)

Backtracking mostly return phase me hota hai.

---

# 🔥 3️⃣ Reverse Stack Using Recursion

Yeh interview favourite hai.

Constraint:

👉 No extra stack allowed.

---

## 🧠 Idea

1. Pop top element
2. Recursively reverse remaining stack
3. Insert popped element at bottom

---

## Step-by-step intuition

Example stack:

```
Top
3
2
1
```

Goal:

```
1
2
3
```

---

## ⭐ Java Code

```
import java.util.Stack;

class Solution {

    // Function to insert element at bottom
    static void insertAtBottom(Stack<Integer> stack, int va
lue) {

        // If stack empty, push directly
        if (stack.isEmpty()) {
            stack.push(value);
            return;
        }

        // Remove top element
        int top = stack.pop();

        // Recursive call
        insertAtBottom(stack, value);

        // Push removed element back
        stack.push(top);
    }

    // Function to reverse stack
    static void reverse(Stack<Integer> stack) {

        if (stack.isEmpty()) return;
```

```
        // Remove top
        int top = stack.pop();

        // Reverse remaining stack
        reverse(stack);

        // Insert removed element at bottom
        insertAtBottom(stack, top);
    }
}
```

---

## 🧠 Why It Works?

Because recursion acts like temporary storage.

Implicit call stack holds removed elements.

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

---

# 🔥 4️⃣ Sort Stack Using Recursion

Goal:

Sort stack in ascending order (top smallest)

---

## 🧠 Idea

1. Pop top element
2. Recursively sort remaining stack
3. Insert popped element at correct position

---

## ⭐ Java Code

```
import java.util.Stack;

class Solution {
```

```
    // Insert element in sorted stack
    static void insertSorted(Stack<Integer> stack, int valu
e) {

        // If stack empty OR top <= value
        if (stack.isEmpty() || stack.peek() <= value) {
            stack.push(value);
            return;
        }

        // Remove top
        int top = stack.pop();

        // Recursive call
        insertSorted(stack, value);

        // Push back removed element
        stack.push(top);
    }

    // Sort stack
    static void sortStack(Stack<Integer> stack) {

        if (stack.isEmpty()) return;

        // Remove top
        int top = stack.pop();

        // Sort remaining stack
        sortStack(stack);

        // Insert removed element correctly
        insertSorted(stack, top);
    }
}
```

---

## 🧠 Deep Insight

Ye pattern actually ye karta hai:

Recursion stack temporarily elements hold karta hai

Aur correct position find karne me help karta hai.

---

# 🎯 Recursion + Stack Relation in Backtracking

Backtracking works like:

1. Choose
2. Explore
3. Undo (backtrack)

Undo ka matlab kya hai?

👉 Call stack unwind hoga

👉 State revert hoga

Exactly stack jaisa.

---

# 🔥 Example: Permutations

When you generate permutations:

- Add element
- Recurse
- Remove element (backtrack)

Ye removal stack nature hai.

---

# 🧠 IMPORTANT INTERVIEW LINE

> Recursion internally uses the system call stack to store function states.
>
> Many stack-based problems can be solved using recursion because recursion itself behaves like a stack.

---

# 🏆 Summary

| Concept | Insight |
|---|---|
| Recursion | Implicit stack |
| Stack DS | Explicit stack |
| Reverse stack | Pop → recurse → insert bottom |
| Sort stack | Pop → recurse → insert sorted |
| Backtracking | Choose → explore → undo |

---

# 1️⃣ Min Stack (LC 155) ⭐ VERY IMPORTANT

## 🧠 Problem

Design stack supporting:

- push()
- pop()
- top()
- getMin()

All in O(1) time.

---

## ❌ Naive Approach

Har push ke baad minimum scan karo

→ O(n) ❌ (Not allowed)

---

## ✅ Optimal Approach: Use 2 Stacks

1. Normal stack → store elements
2. Min stack → store current minimum

---

## 🔥 Logic

Push:

- Normal stack me push
- Min stack me:
  - If empty OR value <= current min → push

Pop:

- If popped value == minStack.peek() → pop from minStack

---

## ⭐ Java Code (Two Stack Approach)

```
import java.util.Stack;

class MinStack {

    private Stack<Integer> stack;
    private Stack<Integer> minStack;

    public MinStack() {
        stack = new Stack<>();
        minStack = new Stack<>();
    }

    public void push(int val) {

        stack.push(val);

        // If minStack empty OR new val <= current min
        if (minStack.isEmpty() || val <= minStack.peek()) {
            minStack.push(val);
        }
    }

    public void pop() {

        int removed = stack.pop();

        // If removed element was minimum
```

```
        if (removed == minStack.peek()) {
            minStack.pop();
        }
    }

    public int top() {
        return stack.peek();
    }

    public int getMin() {
        return minStack.peek();
    }
}
```

Time: O(1)

Space: O(n)

---

## 🎯 Interview Insight

> We maintain an auxiliary stack that keeps track of minimum values dynamically.

---

# 2️⃣ Min Stack WITHOUT Extra Stack (Advanced Trick)

🔥 FAANG follow-up question:

"Can you do it in one stack?"

---

## 🧠 Idea

Store difference instead of actual value.

Keep a variable:

```
min
```

Push:

```
if stack empty:
    push 0
    min = val
else:
    push(val - min)
    if val < min:
        min = val
```

## ⭐ Code

```
import java.util.Stack;

class MinStack {

    Stack<Long> stack;
    long min;

    public MinStack() {
        stack = new Stack<>();
    }

    public void push(int val) {

        if (stack.isEmpty()) {
            stack.push(0L);
            min = val;
        } else {

            stack.push(val - min);

            if (val < min) {
                min = val;
            }
        }
    }
```

```
    public void pop() {

        long diff = stack.pop();

        if (diff < 0) {
            min = min - diff;
        }
    }

    public int top() {

        long diff = stack.peek();

        if (diff >= 0) {
            return (int)(min + diff);
        } else {
            return (int)min;
        }
    }

    public int getMin() {
        return (int)min;
    }
}
```

Space optimized version 🔥

---

# 3️⃣ Implement Queue using Stacks (LC 232)

## 🧠 Idea

Queue:

FIFO

Stack:

LIFO

Use 2 stacks:

- input stack
- output stack

---

## 🔥 Logic

Push:

→ push to input stack

Pop:

→ If output empty:

move all from input → output

→ pop from output

---

## ⭐ Code

```
import java.util.Stack;

class MyQueue {

    Stack<Integer> input;
    Stack<Integer> output;

    public MyQueue() {
        input = new Stack<>();
        output = new Stack<>();
    }

    public void push(int x) {
        input.push(x);
    }

    public int pop() {

        if (output.isEmpty()) {
```

```
            while (!input.isEmpty()) {
                output.push(input.pop());
            }
        }

        return output.pop();
    }

    public int peek() {

        if (output.isEmpty()) {

            while (!input.isEmpty()) {
                output.push(input.pop());
            }
        }

        return output.peek();
    }

    public boolean empty() {
        return input.isEmpty() && output.isEmpty();
    }
}
```

Amortized O(1)

---

# 4️⃣ Implement Stack using Queues (LC 225)

## 🧠 Idea

Make newest element always at front.

Use 1 queue:

Push:

- Add element

- Rotate queue

## ⭐ Code (One Queue Solution)

```
import java.util.LinkedList;
import java.util.Queue;

class MyStack {

    Queue<Integer> queue;

    public MyStack() {
        queue = new LinkedList<>();
    }

    public void push(int x) {

        queue.add(x);

        int size = queue.size();

        // Rotate all previous elements
        for (int i = 0; i < size - 1; i++) {
            queue.add(queue.remove());
        }
    }

    public int pop() {
        return queue.remove();
    }

    public int top() {
        return queue.peek();
    }

    public boolean empty() {
        return queue.isEmpty();
```

```
    }
}
```

Push: O(n)

Pop: O(1)

---

# 🧠 Design Thinking Pattern

| Problem | Core Trick |
|---|---|
| Min Stack | Maintain extra min info |
| Queue via Stack | Reverse order using 2 stacks |
| Stack via Queue | Rotate elements |
| O(1) optimization | Store extra metadata |

---

# 🎯 Interview Master Statements

For Min Stack:

> We maintain an auxiliary structure to track minimum values dynamically in constant time.

For Queue via Stack:

> We reverse the order twice to simulate FIFO using LIFO.

For Stack via Queue:

> We rotate elements to simulate LIFO behavior on FIFO structure.

---

# 🏆 Big Picture

These design problems test:

- Abstraction thinking
- Time complexity awareness
- Data structure transformation

- Amortized analysis

---

# 1️⃣ Trapping Rain Water (LC 42)

## 🧠 Problem

Given heights, calculate trapped water.

Example:

```
[0,1,0,2,1,0,1,3,2,1,2,1]
```

---

## ✅ Approach 1: Two Pointer (Most Optimal)

### 🔥 Intuition

Water depends on:

```
min(leftMax, rightMax) - height[i]
```

Use:

- left pointer
- right pointer
- leftMax
- rightMax

---

## ⭐ Java Code (Two Pointer)

```
class Solution {
    public int trap(int[] height) {

        int left = 0;
        int right = height.length - 1;

        int leftMax = 0;
```

```
        int rightMax = 0;

        int water = 0;

        while (left < right) {

            if (height[left] < height[right]) {

                if (height[left] >= leftMax) {
                    leftMax = height[left];
                } else {
                    water += leftMax - height[left];
                }

                left++;

            } else {

                if (height[right] >= rightMax) {
                    rightMax = height[right];
                } else {
                    water += rightMax - height[right];
                }

                right--;
            }
        }

        return water;
    }
}
```

Time: O(n)

Space: O(1)

---

## ✅ Stack Version (Boundary Detection)

Used when we want container detection.

Already covered earlier but quick recap:

- Maintain increasing stack of indices
- When current height > stack top → container found

---

# 2️⃣ Largest Rectangle in Histogram (LC 84)

## 🧠 Core Idea

For each bar:

Find left smaller and right smaller boundary.

Width:

```
right - left - 1
```

---

## ⭐ One Pass Stack Code

```
class Solution {
    public int largestRectangleArea(int[] heights) {

        Stack<Integer> stack = new Stack<>();
        int maxArea = 0;
        int n = heights.length;

        for (int i = 0; i <= n; i++) {

            int height = (i == n) ? 0 : heights[i];

            while (!stack.isEmpty() && height < heights[sta
ck.peek()]) {

                int h = heights[stack.pop()];
                int right = i;
                int left = stack.isEmpty() ? -1 : stack.pee
k();
```

```
                int width = right - left - 1;
                maxArea = Math.max(maxArea, h * width);
            }

            stack.push(i);
        }

        return maxArea;
    }
}
```

Time: O(n)

---

# 3️⃣ Asteroid Collision (LC 735)

## 🧠 Problem

Positive → right direction

Negative → left direction

Collision only when:

```
top > 0 AND current < 0
```

---

## 🔥 Intuition

Use stack to simulate.

Cases:

- Same direction → push
- Opposite direction → collision

---

## ⭐ Java Code

```
import java.util.Stack;

class Solution {
```

```
    public int[] asteroidCollision(int[] asteroids) {

        Stack<Integer> stack = new Stack<>();

        for (int asteroid : asteroids) {

            boolean destroyed = false;

            while (!stack.isEmpty() &&
                   stack.peek() > 0 &&
                   asteroid < 0) {

                int top = stack.peek();

                if (Math.abs(top) < Math.abs(asteroid)) {
                    stack.pop();  // top destroyed
                    continue;
                }

                else if (Math.abs(top) == Math.abs(asteroi
d)) {
                    stack.pop();  // both destroyed
                }

                destroyed = true;
                break;
            }

            if (!destroyed) {
                stack.push(asteroid);
            }
        }

        int[] result = new int[stack.size()];
        for (int i = stack.size() - 1; i >= 0; i--) {
            result[i] = stack.pop();
        }
```

```
        return result;
    }
}
```

Time: O(n)

---

# 4️⃣ Remove K Digits (LC 402) ⭐ VERY IMPORTANT

## 🧠 Problem

Remove k digits to form smallest number.

Example:

```
num = "1432219"
k = 3
Output = "1219"
```

---

## 🔥 Core Idea

Maintain **Monotonic Increasing Stack**

If current digit < stack top

→ pop (greedy removal)

---

## ⭐ Java Code

```
class Solution {
    public String removeKdigits(String num, int k) {

        Stack<Character> stack = new Stack<>();

        for (char digit : num.toCharArray()) {

            while (!stack.isEmpty() &&
                   k > 0 &&
```

```
                    stack.peek() > digit) {

                stack.pop();
                k--;
            }

            stack.push(digit);
        }

        // If still removals left
        while (k > 0) {
            stack.pop();
            k--;
        }

        // Build result
        StringBuilder sb = new StringBuilder();

        while (!stack.isEmpty()) {
            sb.append(stack.pop());
        }

        sb.reverse();

        // Remove leading zeros
        while (sb.length() > 1 && sb.charAt(0) == '0') {
            sb.deleteCharAt(0);
        }

        return sb.toString();
    }
}
```

Time: O(n)

---

## 🧠 Pattern Summary

| Problem | Pattern Used |
|---|---|
| Rain Water | Two Pointer / Stack |
| Histogram | Monotonic Increasing Stack |
| Asteroid | Collision Simulation |
| Remove K Digits | Monotonic Increasing (Greedy) |

---

# 🎯 Deep Insight

Hybrid problems mean:

👉 Stack for structure

👉 Two pointer for boundary

👉 Greedy removal logic

👉 Collision detection logic

---

# 🏆 Interview Master Statement

> These problems rely on maintaining structural monotonicity or detecting boundary conditions efficiently using stack or two-pointer technique to achieve linear time complexity.

---

---

# 1️⃣ Celebrity Problem

## 🧠 Problem

Party me n log hain.

Celebrity:

- Sab usko jaante hain
- Wo kisi ko nahi jaanta

Matrix:

```
M[i][j] = 1 → i knows j
```

---

## 🔥 Optimal Approach (Stack / Two Pointer Elimination)

### Key Idea:

If A knows B → A cannot be celebrity

If A doesn't know B → B cannot be celebrity

Eliminate until 1 candidate left.

---

## ⭐ Java Code (Two Pointer Version – Best)

```
class Solution {
    int celebrity(int M[][], int n) {

        int a = 0;
        int b = n - 1;

        while (a < b) {

            if (M[a][b] == 1) {
                a++;   // a knows b → a not celebrity
            } else {
                b--;   // a doesn't know b → b not celebrit
y
            }
        }

        // verify candidate
        for (int i = 0; i < n; i++) {

            if (i != a &&
               (M[a][i] == 1 || M[i][a] == 0)) {
                return -1;
            }
        }

        return a;
```

```
    }
}
```

Time: O(n)

---

# 2️⃣ Decode String (LC 394)

Example:

```
"3[a2[c]]"
Output → "accaccacc"
```

---

## 🧠 Pattern

Stack + String building

When `]` appears:

- Pop until `[`
- Repeat substring k times

---

## ⭐ Java Code

```
class Solution {
    public String decodeString(String s) {

        Stack<Integer> countStack = new Stack<>();
        Stack<StringBuilder> stringStack = new Stack<>();

        StringBuilder current = new StringBuilder();
        int k = 0;

        for (char ch : s.toCharArray()) {

            if (Character.isDigit(ch)) {
                k = k * 10 + (ch - '0');
            }
```

```
            else if (ch == '[') {
                countStack.push(k);
                stringStack.push(current);
                current = new StringBuilder();
                k = 0;
            }

            else if (ch == ']') {

                int repeat = countStack.pop();
                StringBuilder prev = stringStack.pop();

                for (int i = 0; i < repeat; i++) {
                    prev.append(current);
                }

                current = prev;
            }

            else {
                current.append(ch);
            }
        }

        return current.toString();
    }
}
```

---

# 3️⃣ Basic Calculator Series

These are from **LeetCode**

---

## 🟢 Basic Calculator I (LC 224)

Operators:

```
+ , - , ()
```

### Pattern:

Sign handling + stack for parentheses

---

## 🟢 Basic Calculator II (LC 227)

Operators:

```
+ , - , * , /
```

No parentheses.

### Pattern:

Monotonic stack for handling precedence.

---

## 🟢 Basic Calculator III (LC 772)

Full calculator:

```
+ - * / ( )
```

### Pattern:

Combine both ideas.

---

## ⭐ Calculator II Clean Code

```
class Solution {
    public int calculate(String s) {

        Stack<Integer> stack = new Stack<>();
        int num = 0;
        char sign = '+';

        for (int i = 0; i < s.length(); i++) {

            char ch = s.charAt(i);

            if (Character.isDigit(ch)) {
```

```
                num = num * 10 + (ch - '0');
            }

            if (!Character.isDigit(ch) && ch != ' '
                || i == s.length() - 1) {

                if (sign == '+') stack.push(num);
                if (sign == '-') stack.push(-num);
                if (sign == '*') stack.push(stack.pop() * n
um);
                if (sign == '/') stack.push(stack.pop() / n
um);

                sign = ch;
                num = 0;
            }
        }

        int result = 0;
        while (!stack.isEmpty())
            result += stack.pop();

        return result;
    }
}
```

---

# 4️⃣ Maximal Rectangle (LC 85)

Matrix of 0/1.

Goal:

Largest rectangle of 1s.

---

## 🔥 Ultra Important Insight

Each row → treat as Histogram

Apply Largest Rectangle in Histogram

---

## ⭐ Java Code

```
class Solution {

    public int maximalRectangle(char[][] matrix) {

        if (matrix.length == 0) return 0;

        int cols = matrix[0].length;
        int[] heights = new int[cols];

        int maxArea = 0;

        for (char[] row : matrix) {

            for (int i = 0; i < cols; i++) {

                if (row[i] == '1')
                    heights[i] += 1;
                else
                    heights[i] = 0;
            }

            maxArea = Math.max(maxArea,
                    largestRectangleArea(heights));
        }

        return maxArea;
    }

    private int largestRectangleArea(int[] heights) {

        Stack<Integer> stack = new Stack<>();
        int maxArea = 0;

        for (int i = 0; i <= heights.length; i++) {

            int h = (i == heights.length) ? 0 : heights[i];
```

```
            while (!stack.isEmpty() &&
                   h < heights[stack.peek()]) {

                int height = heights[stack.pop()];
                int right = i;
                int left = stack.isEmpty() ? -1 : stack.pee
k();

                maxArea = Math.max(maxArea,
                        height * (right - left - 1));
            }

            stack.push(i);
        }

        return maxArea;
    }
}
```

Time: O(rows × cols)

---

# 5️⃣ Sliding Window Maximum (LC 239)

## 🧠 Pattern

Monotonic Decreasing Deque

Why deque?

Because:

- Remove from front (window out)
- Remove from back (maintain decreasing order)

---

## ⭐ Java Code

```java
import java.util.*;

class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {

        Deque<Integer> dq = new LinkedList<>();
        int n = nums.length;
        int[] result = new int[n - k + 1];

        for (int i = 0; i < n; i++) {

            // Remove out of window
            if (!dq.isEmpty() && dq.peek() < i - k + 1)
                dq.poll();

            // Maintain decreasing order
            while (!dq.isEmpty() &&
                   nums[dq.peekLast()] < nums[i]) {
                dq.pollLast();
            }

            dq.offer(i);

            if (i >= k - 1)
                result[i - k + 1] = nums[dq.peek()];
        }

        return result;
    }
}
```

Time: O(n)

---

# 🧠 Advanced Pattern Summary

| Problem | Core Concept |
|---|---|
| Celebrity | Elimination logic |

| Problem | Core Concept |
|---|---|
| Decode String | Nested stack handling |
| Calculator | Operator precedence + stack |
| Max Rectangle | Histogram reuse |
| Sliding Window | Monotonic deque |

---

# 🎯 Interview Insight

Advanced stack problems test:

- Nested structure parsing
- Operator precedence
- Boundary expansion
- Monotonic structure maintenance
- Simulation problems

---